Password Hashing in PHP: Using password_hash and password_verify

Storing passwords securely is critical for protecting user accounts. PHP provides built-in functions, **password_hash** and **password_verify**, to securely hash and verify passwords. These functions use strong algorithms like **bcrypt** by default, making it easy to implement secure password management.

Why Use Password Hashing?

- **Security**: Plain text passwords are vulnerable to theft. Hashing converts passwords into irreversible, fixed-length strings.
- **Protection Against Attacks**: Hashing protects against **brute-force attacks** and **rainbow table attacks**.
- **Best Practice**: Always hash passwords before storing them in a database.

---

1. Hashing Passwords with password_hash

The password_hash function creates a secure hash of a password. It automatically generates a **salt** (random data) and uses a strong hashing algorithm (default: bcrypt).

Syntax

password_hash(string $password, string|int|null $algo, array $options = []): string

- **$password**: The plain text password to hash.
- **$algo**: The hashing algorithm to use (e.g., PASSWORD_DEFAULT or PASSWORD_BCRYPT).
- **$options**: Additional options, such as cost (for bcrypt).

Example

```
$password = "user_password_123";

// Hash the password

$hashed_password = password_hash($password, PASSWORD_DEFAULT);

echo "Hashed Password: " . $hashed_password;
```

Output:

Hashed Password: $2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi

---

2. Verifying Passwords with password_verify

The password_verify function checks if a plain text password matches a hashed password.

Syntax

password_verify(string $password, string $hash): bool

- **$password**: The plain text password to verify.
- **$hash**: The hashed password stored in the database.

Example

```
$password = "user_password_123";

$hashed_password = "$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi";


// Verify the password

if (password_verify($password, $hashed_password)) {

   echo "Password is valid!";

} else {

   echo "Invalid password!";

}
```

Output:

Password is valid!

---

3. Updating Password Hashes

Over time, hashing algorithms may improve, or you may want to increase the **cost** (processing time) of hashing. You can use password_needs_rehash to check if a password needs to be rehashed.

Example

```
$password = "user_password_123";

$hashed_password = "$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi";


// Check if the password needs rehashing

if (password_needs_rehash($hashed_password, PASSWORD_DEFAULT)) {

   $new_hashed_password = password_hash($password, PASSWORD_DEFAULT);

   echo "Password rehashed: " . $new_hashed_password;

} else {

   echo "Password does not need rehashing.";

}
```

---

4. Best Practices for Password Hashing

1. **Use PASSWORD_DEFAULT**:
    - Always use PASSWORD_DEFAULT as the algorithm. It ensures your application uses the strongest available algorithm.
2. **Increase the Cost Factor**:
    - Use the cost option to make hashing slower and more resistant to brute-force attacks.
    -

Example:

```
$options = ['cost' => 12];

$hashed_password = password_hash($password, PASSWORD_DEFAULT, $options);
```

3. **Never Use Weak Algorithms**:
    - Avoid outdated algorithms like MD5 or SHA1. They are not secure for password hashing.
4. **Store Hashes Securely**:

- Store hashed passwords in a secure database column (e.g., VARCHAR(255)).

5. **Validate Input**:
    - Ensure passwords meet complexity requirements (e.g., minimum length, special characters).

---

5. Full Example: User Registration and Login

Registration (Hashing the Password)

```
// User submits a registration form
$username = $_POST['username'];
$password = $_POST['password'];

// Hash the password
$hashed_password = password_hash($password, PASSWORD_DEFAULT);

// Store the username and hashed password in the database
$sql = "INSERT INTO users (username, password) VALUES (?, ?)";
$stmt = $pdo->prepare($sql);
$stmt->execute([$username, $hashed_password]);

echo "User registered successfully!";
```

Login (Verifying the Password)

```
// User submits a login form
$username = $_POST['username'];
$password = $_POST['password'];

// Fetch the user from the database
```

```
$sql = "SELECT password FROM users WHERE username = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$username]);
$user = $stmt->fetch();

if ($user && password_verify($password, $user['password'])) {
   echo "Login successful!";
} else {
   echo "Invalid username or password.";
}
```

---

6. Common Questions

Q: What is the cost option in password_hash?

- The cost option controls the computational cost of hashing. Higher values make hashing slower and more secure but require more server resources.

Example:

```
$options = ['cost' => 12];
$hashed_password = password_hash($password, PASSWORD_BCRYPT, $options);
```

Q: Can I use password_hash for other data?

- While password_hash is designed for passwords, you can use it for other sensitive data. However, consider using encryption for non-password data.

Q: What happens if the hashing algorithm changes?

- If the default algorithm changes (e.g., from bcrypt to argon2), password_verify will still work with old hashes. Use password_needs_rehash to update old hashes.

Summary

Function	Purpose
password_hash	Securely hashes a password using a strong algorithm (e.g., bcrypt).
password_verify	Verifies a plain text password against a hashed password.
password_needs_rehash	Checks if a password needs to be rehashed (e.g., due to algorithm changes).

By using password_hash and password_verify, you can ensure that user passwords are stored securely and protected against common attacks. Always follow best practices for password management to keep your application safe!

in laravel

Password Hashing in Laravel

Laravel simplifies password hashing and verification with its built-in **Hash facade**. Under the hood, Laravel uses PHP's password_hash and password_verify functions, but it provides a cleaner and more expressive syntax. Here's how to use Laravel's password hashing features:

---

1. Hashing Passwords

Laravel provides the Hash facade to hash passwords securely. The default hashing algorithm is **bcrypt**.

Syntax

Hash::make(string $password);

- **$password**: The plain text password to hash.

Example

use Illuminate\Support\Facades\Hash;

$password = "user_password_123";

// Hash the password

$hashed_password = Hash::make($password);

```
echo "Hashed Password: " . $hashed_password;
```

Output:

```
Hashed Password: $2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi
```

---

2. Verifying Passwords

To verify a plain text password against a hashed password, use the Hash::check method.

Syntax

```
Hash::check(string $password, string $hashed_password);
```

- **$password**: The plain text password to verify.
- **$hashed_password**: The hashed password stored in the database.

Example

```
use Illuminate\Support\Facades\Hash;

$password = "user_password_123";
$hashed_password = "$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi";

// Verify the password
if (Hash::check($password, $hashed_password)) {
    echo "Password is valid!";
} else {
    echo "Invalid password!";
}
```

Output:

```
Password is valid!
```

---

3. Rehashing Passwords

If the hashing algorithm or cost factor changes, you may need to rehash passwords. Laravel provides the Hash::needsRehash method to check if a password needs to be rehashed.

Syntax

php

Copy

```
Hash::needsRehash(string $hashed_password);
```

- **$hashed_password**: The hashed password to check.

Example

```
use Illuminate\Support\Facades\Hash;


$hashed_password = "$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi";


// Check if the password needs rehashing
if (Hash::needsRehash($hashed_password)) {
   $new_hashed_password = Hash::make("user_password_123");
   echo "Password rehashed: " . $new_hashed_password;
} else {
   echo "Password does not need rehashing.";
}
```

---

4. Using Password Hashing in Laravel Applications

Registration (Hashing the Password)

When a user registers, hash their password before storing it in the database.

Example:

```
use Illuminate\Support\Facades\Hash;
use App\Models\User;
```

```
// User submits a registration form
$username = $request->input('username');
$password = $request->input('password');

// Hash the password
$hashed_password = Hash::make($password);

// Create a new user
$user = User::create([
    'username' => $username,
    'password' => $hashed_password,
]);

echo "User registered successfully!";
```

Login (Verifying the Password)

When a user logs in, verify their password against the hashed password stored in the database.

Example:

```
php
Copy
use Illuminate\Support\Facades\Hash;
use App\Models\User;

// User submits a login form
$username = $request->input('username');
$password = $request->input('password');
```

```
// Fetch the user from the database
$user = User::where('username', $username)->first();


// Verify the password
if ($user && Hash::check($password, $user->password)) {
    echo "Login successful!";
} else {
    echo "Invalid username or password.";
}
```

---

5. Configuring Hashing in Laravel

Laravel's hashing configuration is located in config/hashing.php. You can customize the hashing algorithm and options here.

Default Configuration

```
'driver' => 'bcrypt', // Default hashing algorithm
'bcrypt' => [
    'rounds' => 10, // Cost factor for bcrypt
],
'argon' => [
    'memory' => 1024, // Memory cost for argon
    'threads' => 2,   // Threads for argon
    'time' => 2,      // Time cost for argon
],
```

Changing the Hashing Algorithm

To use **Argon2** instead of bcrypt, update the driver in config/hashing.php:

'driver' => 'argon',

---

6. Best Practices for Password Hashing in Laravel

1. **Always Hash Passwords**:
    - Never store plain text passwords in the database.
2. **Use Strong Algorithms**:
    - Stick to the default bcrypt or use argon2 for stronger security.
3. **Increase the Cost Factor**:
    - Increase the rounds option for bcrypt or adjust the memory, threads, and time options for argon2 to make hashing more secure.
4. **Validate Input**:
    - Ensure passwords meet complexity requirements (e.g., minimum length, special characters).
5. **Use HTTPS**:
    - Always transmit passwords over HTTPS to prevent interception.

---

7. Common Questions

Q: Can I use Hash::make for other data?

- While Hash::make is designed for passwords, you can use it for other sensitive data. However, consider using encryption for non-password data.

Q: What happens if the hashing algorithm changes?

- Laravel's Hash::check will still work with old hashes. Use Hash::needsRehash to update old hashes if the algorithm or options change.

Q: How do I use Argon2 in Laravel?

- Update the driver in config/hashing.php to argon and configure the argon options as needed.

Summary

Method	Purpose
Hash::make	Securely hashes a password using bcrypt or argon2.
Hash::check	Verifies a plain text password against a hashed password.
Hash::needsRehash	Checks if a password needs to be rehashed (e.g., due to algorithm changes).